

Manual Testing Notes — Easy Language Version

PART 1: SPRINT AND RELEASE (The Basics)

Why do we need Sprints?

Think of building a big shopping app. It needs many things: Login, Signup, Add to Cart, Payment, Order Tracking, Wishlist.

Developers can't build all of this in one day. So they break the work into small pieces. Each small piece of work is called a **Sprint**.

After doing many Sprints, the company finally gives the finished app to customers. Giving the app to customers is called a **Release**.

Simple Definitions

- **Sprint** = A short time (usually 2 weeks) where the team builds and tests a few features.
- **Release** = Giving the final, tested software to real customers.

Easy way to remember: Writing a book → each chapter you finish = a Sprint. Publishing the whole book = the Release.

How long is a Sprint?

Usually 2 weeks. Sometimes 1, 3, or 4 weeks.

What happens inside a Sprint? (Step by step)

Product Backlog → Sprint Planning → Development → Testing
→ Bug Fixing → Retesting → Sprint Review → Sprint Retrospective → Next Sprint

Let's go one by one, in simple words:

1. Product Backlog This is just a big list of everything the app needs. Example: Login, Signup, Search, Payment, Cart.

2. Sprint Planning The team picks a few items from that list to work on this time, and decides who will do what.

3. Development Developers write the actual code.

4. Testing Testers check if the new feature works properly.

5. Bug Reporting If something is wrong, the tester writes down the problem. Example:

```
Bug ID      : BUG-101
Feature     : Login
Expected    : Login should open dashboard
Actual      : App crashes
Priority     : High
Severity    : Critical
```

- **Priority** = How soon this needs to be fixed (business decision)
- **Severity** = How bad the problem is for the app (technical impact)

6. Bug Fixing Developer fixes the reported problem.

7. Retesting Tester checks that same bug again. Either it's fixed (Pass) or still broken (Reopen).

8. Sprint Review The team shows the finished work to the Product Owner or client.

9. Sprint Retrospective The team talks about what went well and what to improve next time.

What happens after many Sprints?

Once enough features are ready (after several sprints), the company decides to release the app.

Release Flow (Simple Steps)

```
Many Sprints Done → Final Testing → Regression Testing
→ UAT (Client Testing) → Approval → Deployment → Release → Maintenance
```

- **Final Testing** = Testing the whole app together (not just one feature)
- **Regression Testing** = Making sure old features still work after new changes
- **UAT** = The client or business people test the app themselves
- **Deployment** = Putting the app on the live server
- **Release** = Now customers can use it
- **Maintenance** = Fixing bugs and adding small improvements after release

Sprint vs Release — Simple Comparison

Sprint	Release
Small piece of work	Final delivery to customers
Takes 1-4 weeks	Happens after many sprints
Builds a few features	Has many features ready together
Only the team sees it	Customers see it
Can have small bugs (fixed during the sprint)	Should be stable, no major bugs

Important point for interviews: Not every sprint gets released. A company usually waits for many sprints to finish, then releases everything together.

PART 2: HOW CODE ACTUALLY REACHES THE TESTER (Real Company Process)

Let's understand this like a simple story.

Step-by-step story

- 1. Customer tells what they want.** "I want a shopping website with login, cart, payment."
- 2. A Business Analyst (BA) writes it down properly.** This document is called BRD or SRS — just a fancy name for "list of requirements."
- 3. Sprint Planning happens.** The team picks: "This time, let's build Login and Signup."
- 4. Developer writes the code.** Most apps are built in **3 layers**:

```
UI Layer (what user sees, like the login screen)
    ↓
Business Logic Layer (checks: is password correct?)
    ↓
Database Layer (stores and reads user data)
```

- 5. Another developer checks the code (Code Review).** Just to make sure it's written properly, with no obvious mistakes.
- 6. Developer saves the code to Git.** Git is just an online storage place for code. Common types: GitHub, GitLab, Bitbucket, Azure DevOps.
- 7. DevOps or a tool automatically builds the app.** "Build" means turning the code into something that can actually run — like a ZIP file, an APK (for mobile apps), a JAR/WAR file, or an EXE.

8. The build goes to a testing server. In most modern companies, this happens automatically — the tester just opens a website link (like a test URL) and the new version is already there. No manual ZIP sharing needed.

(In small or older companies, sometimes the ZIP/APK file is shared manually through email or a shared folder, and the tester downloads and installs it themselves. Both ways exist — but automatic is more common today.)

9. Smoke Testing — a quick check first. Before doing full testing, the tester just checks: Does the app even open? Can I log in? If this fails, the build is rejected immediately, and developers need to fix it first.

10. Functional Testing — the real testing begins. Now the tester checks every feature properly against what was asked for.

11. If a bug is found, it's reported. Usually in a tool like Jira. Example:

```
Bug ID    : BUG-105
Module    : Login
Expected  : Dashboard should open
Actual    : App crashes
Priority   : High
Severity  : Critical
```

12. Team Lead checks the bug. Makes sure it's a real problem and gives it to the right developer.

13. Developer fixes it and saves the new code.

14. A new build is created (like version 1.0.1).

15. Tester checks the same bug again (Retesting). Fixed → Pass. Still broken → Reopen.

16. Tester also checks nearby features (Regression Testing). Just to make sure the fix didn't accidentally break something else, like Signup or Logout.

17. Meanwhile, developers don't wait around. While testers are still testing Sprint 1, developers usually already start working on Sprint 2. This is how the whole process keeps moving fast.

18. After many sprints are done, final testing happens for the whole app.

19. UAT — the client tests it themselves.

20. If approved, the app is deployed to Production (the live server).

21. Now customers can use the new version.

22. Maintenance continues — small bug fixes and improvements after release.

Quick Summary Flow

Customer Requirement → BA writes requirements → Sprint Planning
→ Developer writes code → Code Review → Save to Git
→ Build created automatically → Sent to test server
→ Smoke Test → Full Testing → Bug found? → Report bug
→ Bug fixed → New build → Retest → Regression Test
→ Sprint finished → Next sprint starts
→ (after many sprints) Final Testing → UAT → Approval
→ Live for customers → Maintenance continues

PART 3: THREE TESTING TECHNIQUES (Easy Explanation)

These are simple methods testers use to pick **which test cases to write**, so they don't waste time testing everything randomly.

3.1 What is Specification-Based Testing?

Simple meaning: You write test cases just by reading the requirement document — you never look at the code. You only care about: "If I give this input, what output should I get?"

This is what testers normally do, because most testers never see the actual code. They just read what the app is *supposed* to do, and test if it really does that.

3.2 Equivalence Partitioning (EP) — Easy Explanation

The problem: Suppose a password field must be 8 to 15 characters long. There are hundreds of possible lengths. Testing all of them would take forever.

The solution (EP): Group similar values together, and just test **one value from each group**. If one value works in a group, we assume the rest of that group behaves the same way.

Example: Password must be 8-15 characters

Group	Example Value	What Should Happen
Too short	6 characters	Rejected
Correct length	10 characters	Accepted
Too long	17 characters	Rejected

Just **3 tests** cover the whole rule, instead of testing every possible length.

Another example: Age must be between 18 and 60

Group	Example	Expected Result
Too young	10	Rejected
Correct age	30	Accepted
Too old	70	Rejected
Not even a number	"abc"	Rejected

In simple words: EP saves your time by not testing every single value — just one from each group is enough.

3.3 Boundary Value Analysis (BVA) — Easy Explanation (Bonus)

Most bugs hide right at the edges of a rule, not in the middle. So along with EP, we also test the exact border numbers.

Example: Age must be 18 to 60

Test Value	What It Checks	Expected
17	Just below the limit	Rejected
18	Exact starting point	Accepted
60	Exact ending point	Accepted
61	Just above the limit	Rejected

In simple words: EP tells you which groups to test. BVA tells you to test right at the edges of those groups, because that's where mistakes usually happen.

3.4 Decision Table Testing — Easy Explanation

When to use this: When the result depends on **more than one condition together**, not just one field.

Example: Discount Rule

Rule:

- If customer IS a member AND order is more than ₹1000 → 20% discount
- If member but order is ₹1000 or less → 10% discount
- If not a member but order is more than ₹1000 → 5% discount
- If not a member and order is ₹1000 or less → No discount

We make a simple table with all combinations:

Is Member?	Order > ₹1000?	Discount
Yes	Yes	20%
Yes	No	10%
No	Yes	5%
No	No	0%

Each row = one test case. This way, **no combination is missed**.

Another Example: Login Rule

Rule:

- Correct username + Correct password + Account not locked → Login Success
- Correct username + Correct password + Account locked → "Account Locked" message
- Wrong username or wrong password → "Invalid Credentials" message

Username Correct?	Password Correct?	Account Locked?	Result
Yes	Yes	No	Login Success
Yes	Yes	Yes	Account Locked Message
No or Wrong	—	—	Invalid Credentials Message

In simple words: Decision Table is used when there are multiple "if this AND that" rules together, so you don't accidentally forget to test a combination.

3.5 Where do these techniques fit in the real process?

Remember Step 10 from Part 2 — "Functional Testing." That is exactly where you use these techniques:

Build is ready on test server

↓

Smoke Test (quick check)

↓

Functional Testing ← Use EP, BVA, and Decision Table HERE

↓

Bug found → Report → Fix → Retest → Regression Test

Quick Comparison Table

Technique	When to use	What it does
Equivalence Partitioning	One field with a range of values	Groups similar values, test one from each group
Boundary Value Analysis	Same field, but focus on edges	Tests the exact min/max border values
Decision Table	Many conditions combine together	Makes sure every combination is tested

PART 4: SIMPLE INTERVIEW ANSWERS TO REMEMBER

Q1. What is a Sprint? A short time period (usually 2 weeks) where the team builds and tests a few features.

Q2. What is a Release? Giving the finished, tested app to real customers.

Q3. Does every sprint get released? No. Many sprints are usually combined together before one release.

Q4. What is Equivalence Partitioning? Grouping similar input values together and testing just one value from each group, instead of testing everything.

Q5. What is Boundary Value Analysis? Testing the exact edge values of a rule (like the smallest and largest allowed number), because bugs often hide there.

Q6. What is Decision Table Testing? A table that lists all combinations of multiple conditions and their expected results, so no combination gets missed.

Q7. Difference between EP and Decision Table? EP looks at **one field at a time**. Decision Table looks at **many conditions together**.